

Unified Token Lifecycle Framework

While there is no "correct" classification of a token lifecycle into stages, I have classified them into five steps for ease of comparison. Stage 1: Encoding Stage 2: Attention (within a block) Stage 3: FFN + nonlinear transformation (within a block) Stage 4: Trajectory across depth Stage 5: Output

1. **Initialization & Conditioning (Encoding):** This stage represents the genesis of the token. It focuses on translating raw discrete or continuous data into a unified latent space, augmented with the necessary structural metadata required for the specific architecture to understand its place in space or time.
 - *Core Function:* Embedding generation, positional encoding, and the injection of global conditional priors (e.g., diffusion timesteps or system prompts).
2. **Sequence & Spatial Mixing (Interaction):** This stage governs how a token gathers context from its environment. It focuses entirely on inter-token communication, defining the mechanism through which a token aggregates external signals from other tokens in the sequence or spatial grid.
 - *Core Function:* Information routing across the sequence, context assimilation, and the updating of a token's representation based on its neighbors or a compressed hidden state.
3. **Transformation & Conditional Routing (Processing):** Once a token has gathered context from its environment, this stage governs its internal, intra-token refinement. It involves non-linear feature expansion and, crucially, the decision-making process regarding *where* or *how* the token should be processed next based on its current state.
 - *Core Function:* Independent token transformation, feature extraction, and conditional pathway selection (routing/gating) through specialized sub-networks.
4. **Depth & Temporal Evolution (Integration):** This stage models the cumulative trajectory of the token. Rather than looking at a single layer, this stage captures the token's lifecycle as an ongoing dynamical process, tracking how its representation updates, stabilizes, or denoises as it moves through network depth or iterative temporal steps.
 - *Core Function:* Residual accumulation, state-space transitions over continuous depth, and iterative signal-to-noise refinement.
5. **Projection & Emission (Output):** The final stage of the lifecycle. The matured token representation is decoded from the high-dimensional latent space back into the target observable space, finalizing its journey.

- *Core Function:* Dimensionality reduction, final layer normalization, and mapping to a probability distribution (e.g., vocabulary logits) or continuous value (e.g., noise estimation or pixel values).

Token Lifecycle Framework

Before any LLMs understand text, text is converted into numbers through a process known as tokenization. While there are many distinct tokenization methods, efficient tokenization methods reduce the computing power needed for training and inference. Additionally, tokens (not bits) are the fundamental unit of semantic information. LLMs act as discrete-time channels, and the semantic information flow (directed information density) naturally progresses and accumulates over the token sequence. By analyzing token behavior across each LLM infrastructure, we can analyze each current LLM infrastructure more deeply.

1 Transformer

Fig 1. illustrates a basic transformer architecture. Input is encoded, then passed through the transformers many layer. Most notably, each layer has a multi head attention and a feed forward network. At the end of they layer, the final vector is decoded and passed as output.

1.1 Stage 1: Initialization & Conditioning (Encoding)

Not all transformers have encoders. However, since the original transformer infrastructure introduced had an encoder-decoder structure, it will be included in this token lifecycle overview.

The lifecycle begins by mapping discrete inputs into a continuous, high-dimensional space where they can be mathematically manipulated. Raw tokens are converted into continuous vectors via learned **Embeddings**. Embeddings help capture relationship and context. For instance, related terms like “cat” and “dog” will have closer vector distances.

Because the architecture processes tokens in parallel without inherent sequential logic, positional encodings (such as sinusoidal waves or RoPE) are injected to provide order (information on the position of each word). Tokens are initially projected into a high-dimensional embedding space, typically around 10^3 dimensions.

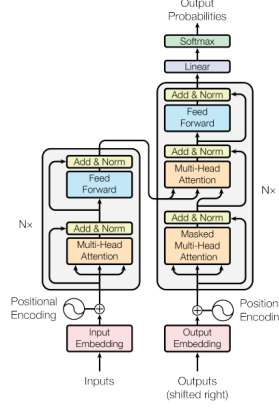


Figure 1: Full Lifecycle of Token in the Transformer Architecture

1.2 Stage 2: Sequence & Spatial Mixing (Interaction)

This stage governs how a token gathers context from its environment, defining the mechanism through which a token aggregates external signals from other tokens in the sequence.

Explicit Dense Interaction The first major component inside each replicated layer of the transformer is the multi-headed attention. This is arguably the most influential component when it comes to identifying and incorporating meaningful information about a token's role in the entire sequence. Multiple heads in this attention mechanism are each specialized in capturing different linguistic aspects simultaneously. The core mechanism is explicitly calculated via self-attention weights, resulting in $\mathcal{O}(n^2)$ computational complexity.

At the heart of every Transformer layer is the scaled dot-product attention mechanism. For a sequence of tokens, the attention weights are calculated by taking the queries (Q) and keys (K), computing their dot product, and passing them through a softmax function:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d}}\right) V$$

To see what happens to an individual token, we can look at the specific attention weight (α_{ij}) that a token i (the query) assigns to another token j (the key):

$$\alpha_{ij} = \frac{\exp(q_i \cdot k_j / \sqrt{d})}{\sum_{m=1}^N \exp(q_i \cdot k_m / \sqrt{d})}$$

Three Phase Model Historically, we might have assumed that each multi-attention layer does roughly the same thing. However, recent research have

shown that tokens go through different processes across different attention layer. This can be divided into three phases: Mix, Compress, and Refine. These phases will be elaborated in different stages of the Unified Token Lifecycle for transformers.

Mix (Early Layers) However, the model actually gathers all possible context, computing how every word relates to the others in a “messy” initial phase. This initial mixing stage is deliberately limited to prevent over-mixing and representational collapse that would occur with extended uniform attention. This controlled, brief mixing phase establishes the semantic foundation that subsequent phases refine.

Attention Sinks and Geometric Forces During this mixing phase, massive token activations cause the model to direct a disproportionate amount of attention to the very first few tokens in a sequence, even if those initial tokens contain no semantic meaning (Attention Sinks). Furthermore, the softmax attention matrix applies an inescapable, attractive force on the tokens. By averaging values, tokens constrained to a hypersphere rapidly lose their angular dispersion and are driven to collapse into a single mass unless countered.

1.3 Stage 3: Transformation & Conditional Routing (Processing)

Once a token has gathered context, this stage governs its internal, intra-token refinement. Tokens pass through Feed-Forward Networks (FFN) to undergo non-linear feature expansion. The FFN step is applied independently on each token, refining the contextual patterns already integrated to yield useful “knowledge” from them. These layers are also supplemented with residual connections and layer normalizations. As a result, at the end of this step, we have a transformer layer with an updated representation that will be input to the next transformer layer.

Compress (Middle Layers) The model squishes the accumulated information into a dense, high-energy summary, creating a **Compression Valley**. During this phase, while effective models have a high-dimensional working space where features are captured, tokens in such models can also be represented in approximately 10-dimensional submanifolds that closely resemble human semantic spaces.

1.4 Stage 4: Depth & Temporal Evolution (Integration)

This stage models the cumulative trajectory of the token, tracking how its representation updates, stabilizes, or denoises as it moves through network depth.

The Residual Stream & Information Bottlenecks This stream is the sum of the outputs from all previous layers and the original embedding. It functions as an internal communication channel, allowing the model to preserve a version history of important events across depth. However, because a token must compress all historical data into its current layer’s residual stream to pass context forward, it suffers from an information bottleneck. Passing through successive self-attention blocks can cause severe feature homogenization, gradually stripping tokens of their unique semantic features.

Transformers as Continuous Flows: The ODE Interpretation The discrete, layer-by-layer update of a transformer has a precise mathematical analogue in continuous dynamical systems. Fein-Ashley (2025) formalizes this by showing that each residual layer update—where a token representation x_n is updated as:

$$x_{n+1} = x_n + \frac{1}{N}f(x_n)$$

—is structurally identical to a **forward Euler step**, the simplest numerical method for solving ordinary differential equations. Here, $f(x) = g(x) + h(x)$ decomposes into the self-attention sublayer g and the feedforward sublayer h , and N is the total number of layers.

As N grows, the **Transformer Flow Approximation Theorem** establishes that token trajectories converge uniformly at rate $\mathcal{O}(1/N)$ to the unique solution of the continuous ODE:

$$\frac{dx(t)}{dt} = f(x(t)), \quad x(0) = E(\text{input})$$

A key consequence emerges when the mapping f satisfies a **one-sided Lipschitz condition** with a negative constant λ :

$$\langle f(x) - f(y), x - y \rangle \leq \lambda \|x - y\|^2, \quad \lambda < 0$$

Under this condition the dynamics are contractive: any perturbation δ_0 between two token representations decays exponentially with depth:

$$\|\delta(t)\| \leq \|\delta_0\| \cdot e^{\lambda t}$$

This is the formal basis for the empirical observation that deep transformers are robust to noise—small input variations are smoothed out as representations propagate through layers. However, contractivity is indiscriminate. The same force that suppresses noise also suppresses semantic distinctions. When λ is strongly negative, diverse token representations are pulled together regardless of meaning, risking **representation collapse**.

Note: Fein-Ashley (2025) was withdrawn by its author in May 2025. The theoretical framework is consistent with the broader Neural ODE literature, but the specific proofs should be treated with appropriate caution pending clarification of the withdrawal.

1.5 Stage 5: Projection & Emission (Output)

The final stage ends the token’s internal journey. The matured token representation is decoded from the high-dimensional latent space back into the target observable space.

Refine (Final Layers) The model shifts from macro-context to micro-prediction, polishing the dense summary to focus purely on the next specific word. From a token-centric perspective, the uniformity breaks. The tokens stop sharing that identical global summary and diverge rapidly. Each token’s vector transforms into a highly specific, individualized pointer meant to trigger the exact vocabulary prediction for whatever word comes next.

Output The final token representations undergo a Final LayerNorm, followed by a Linear projection and a Softmax function. During this final projection, models utilize distinct, family-specific decision strategies that balance expansion (considering different possibilities) and pruning (removing unlikely words) to finalize the prediction. This final mechanical process generates a probability distribution over the entire vocabulary to predict the next token autoregressively, completing the token lifecycle.